10

Управление объектами схем

Copyright © 2006, Oracle. All rights reserved.

Рассматриваемые вопросы

Типы таблиц
Секционирование
IOT
Кластер
Задачи АБД

- Использование таблиц определенных типов в соответствие с вашими требованиями: обычных, секционированных, IOT и кластеризованных.
- Выполнение задач АБД:
  - Оценка размера новых таблиц
  - Анализ тенденций роста
  - Сопровождение статистик оптимизатора
  - Оперативная реорганизация объектов схем

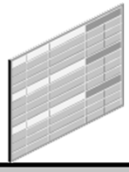
ORACLE

Copyright © 2006, Oracle. All rights reserved.

Типы таблиц

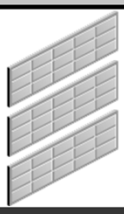
Обычная (heap)

Кластеризованные



Тип таблицы	Описание
Обычная (неупорядоченная)	Данные хранятся неотсортировано в виде груды (heap).
Секционированная	Данные разделены на меньшие, лучше управляемые части.
Индекс-таблица (IOT)	Данные (включая неключевые значения) отсортированы и хранятся в структуре индексного B-дерева.
Кластеризованная	Связанные данные нескольких таблиц хранятся вместе.

Секционированная



Индекс-таблица (IOT)

ORACLE

Copyright © 2006, Oracle. All rights reserved.

Типы таблиц

Обычные (организованные в виде “груды”, “heap-organized”) таблицы были рассмотрены в курсе *Oracle Database 10g: Администрирование I*.

Секции (*partitions*) – части таблицы или индекса, создаваемые для облегчения управления *очень большими базами данных* (*very large database,* VLDB), которые могут содержать несколько терабайтов данных.

Данные таблиц, организованных в виде “груды” (heap-organized table), хранятся как неупорядоченные коллекции (груды или кучи, heap), В отличие от них в и*ндекс-таблице* (*index-organized table, IOT*) данные хранятся в структуре индексного B-дерева, отсортированные по главному ключу..

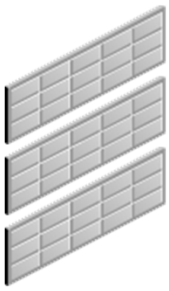
Кластер (*cluster*) – это группа из одной или нескольких таблиц, совместно использующих одни и те же блоки данных, поскольку эти таблицы содержат общие столбцы и часто используются вместе друг с другом.

Что такое секция и для чего она используется?

Типы таблиц
> Секционирование
IOT
Кластер
Задачи АБД

Секция:

- **Часть “очень большой” таблицы или индекса**
- **Хранится в своем собственном сегменте**
- **Используется для повышения производительности и улучшения управляемости**



ORACLE

10-4 Copyright © 2006, Oracle. All rights reserved.

Что такое секция и для чего она используется?

Секция – часть (piece) “очень большой” таблицы или индекса, хранящаяся в своем собственном сегменте. Поэтому такой секцией можно отдельно управлять. Примером “очень большой” таблицы является таблица хранилища данных, содержащая несколько гигабайтов данных. Секции могут быть разбиты дополнительно на *подсекции* (*subpartitions*) для повышения уровня управляемости и улучшения производительности.

Секционирование может способствовать улучшению производительности, так как многие запросы получают возможность игнорировать секции, не соответствующие предложению `WHERE` и поэтому не содержащие запрашиваемые строки. В итоге уменьшается объем полностью просматриваемых данных при получении *результирующего набора* (*result set*).

Операции над секционированными таблицами и индексами могут быть выполнены в параллель. В этом случае различным параллельно выполняемым серверным процессам выделяются для обработки различные секции таблицы или индекса.

Секции

Характеристики секций:

- **Одни и те же логические атрибуты: одинаковые столбцы, ограничения и индексы**
- **Различные физические атрибуты: хранение в разных табличных пространствах**
- **Прозрачны для приложений**
- **Несколько методов секционирования**

ORACLE

10-5 Copyright © 2006, Oracle. All rights reserved.

Секции

Каждая секция хранится в своем собственном сегменте и ей можно управлять отдельно. Она может использоваться независимо от других секций. Секция – структура хранения, позволяющая произвести лучшую настройку для обеспечения доступности и производительности.

В основе одной из возможностей параллельного выполнения лежит использование секций. Операции над секционированными таблицами и индексами могут быть выполнены в параллель. В этом случае различным параллельно выполняемым серверным процессам выделяются для обработки различные секции таблицы или индекса

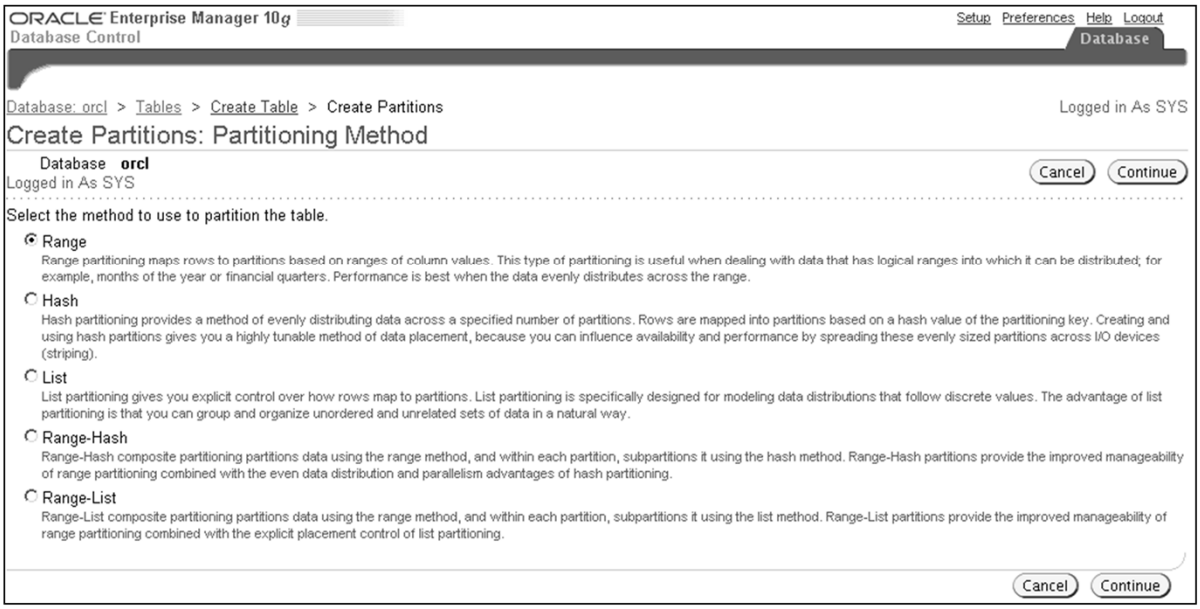
Секции и подсекции таблицы или индекса совместно разделяют одни и те же логические атрибуты. Например, все секции (или подсекции) в таблице разделяют определения одних и тех же столбцов и ограничений, и все секции (или подсекции) индекса разделяют одни и те же опции индекса. Однако физические атрибуты (например, `TABLESPACE`) могут отличаться. Хранение секций в отдельных табличных пространствах полезно для независимого резервирования и восстановления, контроля установления соответствия данных с дисковыми драйверами (балансировка ввода-вывода), а также для снижения вероятности повреждения данных.

Секционирование прозрачно для существующих приложений и обычных команд DML, выполняемых для секционированных таблиц. Однако при программировании приложения можно также воспользоваться преимуществами от использования в командах DML *имен таблиц или индексов с указанием секций* (*partition-extended table or index names*).

Дополнительные сведения о секционированных таблицах и индексах, включая методы

секционирования, см. в документе *Oracle Database Administrator's Guide.*

Создание секции



ORACLE

10-6 Copyright © 2006, Oracle. All rights reserved.

Создание секции

Database Control поддерживает различные типы секционирования таблиц и индексов и упрощает процесс их создания. Мастера (wizards) ведут вас по шагам процесса создания и предлагают возможные опции.

Выберите Administration > Tables. На странице Tables щелкните на кнопке Create, чтобы создать секционированную таблицу. Выводится страница “Create Table: Table Organization”, на которой можно выбрать создание неупорядоченной таблицы (heap-organized table) или индекс-таблицы (index-organized table). Щелкните на кнопке Continue для перехода на страницу Create Table. На этой странице указывается имя таблицы и другая информация. Щелкните на закладке Partitions, а затем на кнопке Create. Открывается страница Create Partitions: Partitioning Method, приведенная на слайде. Она содержит точки входы в мастера создания секционированных таблиц. Примите решение о типе секционирования, а затем щелкните на кнопке Continue. Мастер ведет вас по шагам процесса создания , в ходе которого задаются столбцы, определяющие секции и их спецификации.

Методы секционирования

- **Range: соответствие строк секциям устанавливается на основе логических диапазонов значений столбцов (например, месяцев года)**
- **Hash: строки распределяются по секциям на основе хэш-значения ключа секционирования**
- **List: строки распределяются по секциям на основе задаваемых АБД списков дискретных значений**
- **Range-hash: строки распределяются на основе метода диапазонов на секции, а внутри каждой такой секции на подсекции с использованием хэш-метода**
- **Range-list: строки сначала распределяются на основе метода диапазонов, а затем с использованием дискретных значений**

ORACLE

10-7 Copyright © 2006, Oracle. All rights reserved.

Методы секционирования

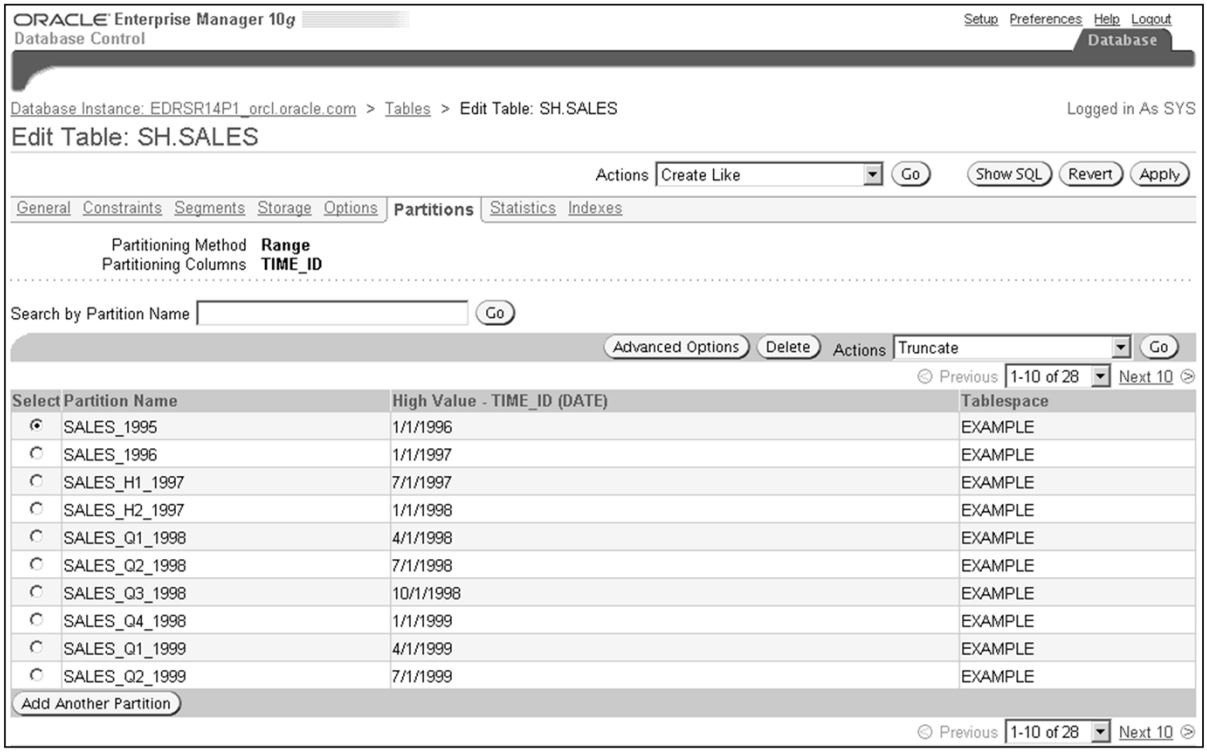
- Производительность при использовании секционирования по диапазонам (**range** partitioning) наилучшая, когда данные равномерно распределены по этим диапазонам.
- Хэш-секционирование (**hash** partitioning) означает автоматическое равномерное распределение данных: АБД не может управлять тем, как конкретная строка попадает в секцию. Пример:

```
CREATE TABLE regions
(region_id NUMBER, region_name VARCHAR2 (25))
PARTITION BY HASH (region_id)
PARTITIONS 4 STORE IN (tbs1, tbs2, tbs3, tbs4);
```

  В этом примере показана хэш-секционированная таблица. Распределение по секциям производится на основе столбца `region_id`. Создается четыре секции, им присваиваются имена, сгенерированные системой, сами секции располагаются в табличных пространствах `tbs1`, `tbs2`, `tbs3`, and `tbs4`.
- Когда необходим полный контроль над распределением строк по секциям, задавайте в описании каждой секции списки (**list**) дискретных значений для столбца, на основе которого осуществляется секционирование. Например, это может быть код штата.
- Смешанное секционирование на основе метода **range-hash** идеально подходит для хронологических (historical) и расщепляемых (striping) данных.
- Секции, созданные на основе метода **range-list**, – это только логические

структуры; их данные хранятся в сегментах подсекций.

Сопровождение секций



ORACLE

10-8 Copyright © 2006, Oracle. All rights reserved.

Сопровождение секций

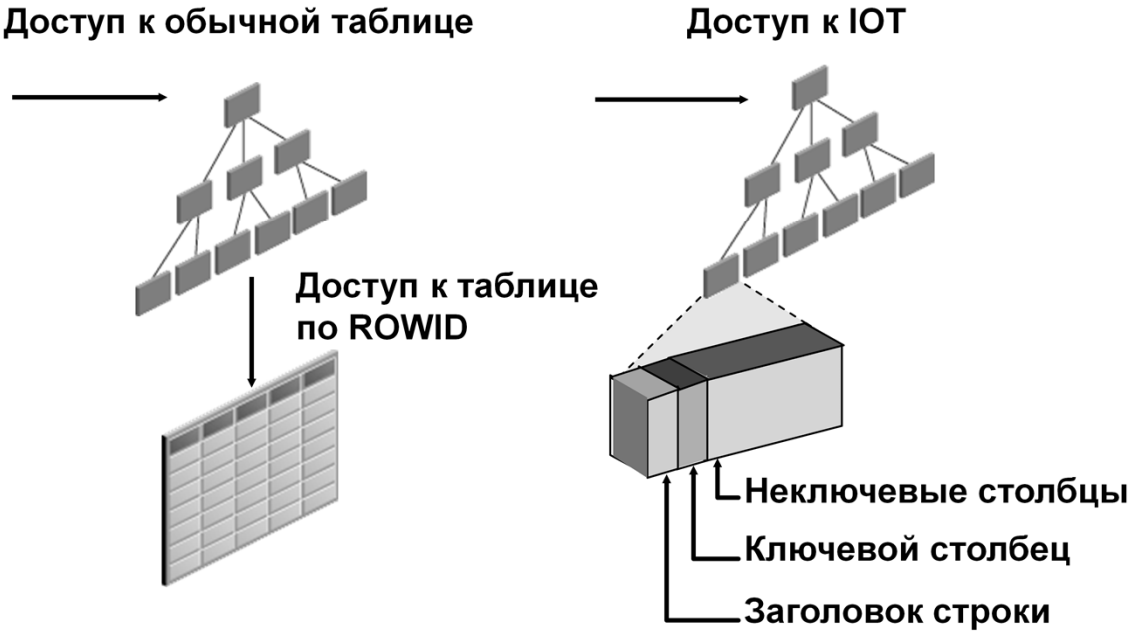
После того, как секционированный объект был создан, его сопровождение можно осуществлять с использованием Database Control.

Выберите Administration > Tables. Щелкните на требуемой таблице. В результате выводится страница Edit Table. После щелчка на закладке Partitions открывается страница, показанная на слайде. На ней можно отметить требуемую секцию и применить к ней операцию сопровождения, которая выбирается в выпадающем списке поля Actions.

Примечание: для изменения характеристик хранения выбранной секции используйте кнопку Advanced Options.

Индекс-таблицы

Типы таблиц
Секционирование
> IOT
Кластер
Задачи АБД



Copyright © 2006, Oracle. All rights reserved.

Индекс-таблицы

В отличие от *обычной неупорядоченной* (*heap-organized*) таблицы в и*ндекс-таблице* (*index-organized table, IOT*) данные хранятся в индексной структуре в виде B-дерева, отсортированные по главному ключу. Кроме значений столбцов первичного ключа в B-дереве IOT также хранятся неключевые столбцы.

Индекс-таблицы поддерживают все возможности, предоставляемые таблицами, например, ограничения, триггеры, столбцы объектного типа и с типом LOB, секционирование, параллельные операции, реорганизацию в оперативном режиме и репликации. Для индекс-таблицы даже можно создавать индексы.

Индекс-таблицы идеально подходят для приложений OLTP, которые требуют наличие быстрого доступа к данным с использованием главного ключа и высокий уровень доступности. Запросы и операции DML над таблицей заказов, используемые при оперативной обработке заказов, основаны преимущественно на доступе по главному ключу. Использование интенсивных операций DML вызывает фрагментацию и, следовательно, необходимость частых реорганизаций. Поскольку индекс-таблица может быть реорганизована в оперативном режиме без перевода в недействительное состояние вторичных индексов, временное окно недоступности данных значительно сокращается.

Индекс-таблица – это альтернатива для:

- таблицы с индексом по главному ключу, созданному по команде `CREATE INDEX`;
- таблицы, хранимой в индексном кластере, который был создан по команде `CREATE CLUSTER` и в котором устанавливается соответствие главного ключа таблицы и ключа кластеризации.

Индекс-таблицы и обычные неупорядоченные таблицы

- **По сравнению с обычной таблицей IOT:**
  - **Обеспечивает более быстрый доступ к табличным данным, который основан на использовании ключа**
  - **Не содержит в двух структурах хранения значение главного ключа**
  - **Требует меньше пространства для хранения**
  - **Использует вторичные индексы и логические ROWID**
  - **Предоставляет более высокий уровень доступности, так как реорганизация таблицы не переводит вторичные индексы в недействительное состояние**
- **IOT имеют следующие ограничения :**
  - **Должны иметь главный ключ, проверка которого не может быть отложенной (DEFERRABLE)**
  - **Не могут входить в кластер**
  - **Нельзя использовать смешанное секционирование**
  - **Нельзя создавать столбцы с типом ROWID или LONG**

ORACLE

10-10 Copyright © 2006, Oracle. All rights reserved.

Индекс-таблицы и обычные неупорядоченные таблицы

Индекс-таблицы не имеют обычных (физических) ROWID. Вместо них используются *логические* ROWID (*logical rowids*). Такие ROWID предоставляют более быстрый доступ к данных в индекс-таблице с использованием двух методов:

- использование *оценки физического расположения* (*physical guess*); при этом время доступа эквивалентно времени доступа по физическим ROWID;
- доступ без оценки (или после неправильной оценки); такие действия выполняются подобно доступу по главному ключу индекс-таблицы.

Оценка (guess) основывается на знании о файле и блоке, в котором находится запись. Такая информация верна, когда индекс создается, но изменяется, если листовой блок расщепляется. Если оценка неправильна и строка больше не размещается в указанном блоке, тогда для доступа к строке используется оставшаяся часть логического ROWID, содержащая главный ключ.

Oracle создает *вторичные индексы* (*secondary indexes*) для индекс-таблицы, используя логические ROWID, основанные на главном ключе таблицы. Так как строки индекс-таблицы не имеют постоянного физического адреса, оценки физического расположения могут устареть после перемещения строк в новые блоки.

Для обновления оценок необходимо перестроить вторичный индекс. Следует отметить, что перестройка вторичного индекса требует чтения базовой таблицы в отличие от перестройки индекса обычной таблицы.

Индекс-таблицы и обычные неупорядоченные таблицы (продолжение)

Более быстрый и простой способ исправления оценок (guesses) – использование команды `ALTER INDEX ... UPDATE BLOCK REFERENCES`. Она выполняется в оперативном режиме и операции DML разрешены над соответствующей индекс-таблицей.

После перестройки вторичного индекса или любых других изменениях ссылок на блоки в оценках, необходимо снова собрать индексные статистики.

Тип данных `UROWID` позволяет использовать в приложениях логические `ROWID` таким же образом, как используются физические `ROWID`, например, как часть курсора или для выборки `ROWID` с целью последующего внесения изменений в данные. `UROWID` может также использоваться для хранения `ROWID` других баз данных, доступных через шлюзы. Наконец, тип `UROWID` может также использоваться для ссылки на физические `ROWID`.

Создание индекс-таблиц

```
SQL> CREATE TABLE country
  2    ( country_id      CHAR(2)
  3      CONSTRAINT country_id_nn NOT NULL,
  4      country_name    VARCHAR2(40),
  5      currency_name   VARCHAR2(25),
  6      currency_symbol VARCHAR2(3),
  7      map             BLOB,
  8      flag            BLOB,
  9      CONSTRAINT country_c_id_pk
 10        PRIMARY KEY (country_id))
 11    ORGANIZATION INDEX
 12    TABLESPACE indx
 13    PCTTHRESHOLD 20
 14    OVERFLOW TABLESPACE users;
```

ORACLE

10-12 Copyright © 2006, Oracle. All rights reserved.

Создание индекс-таблиц

Как правило, индексные записи обычного индекса в виде B-дерева имеют малый размер. В них содержится значение главного ключа и `ROWID` для каждой строки. В листовом блоке может храниться большое количество таких индексных записей. В случае индекс-таблиц такая ситуация не всегда повторяется, поскольку в них строки хранятся целиком.

Хранение больших по размеру записей в индексных листовых блоках замедляет поиск и полный просмотр индекса. Поэтому может быть задано пороговое значение, представляющее собой процент от размера блока, при превышении которого строка перемещается в область переполнения.

Конечно, столбец главного ключа должен всегда храниться в индексном блоке IOT, чтобы выполнялся поиск. Но неключевые значения могут располагаться в отдельной области, называемой *областью переполнения* (*overflow area*), так чтобы само по себе B-дерево оставалось плотно кластеризованным (densely clustered).

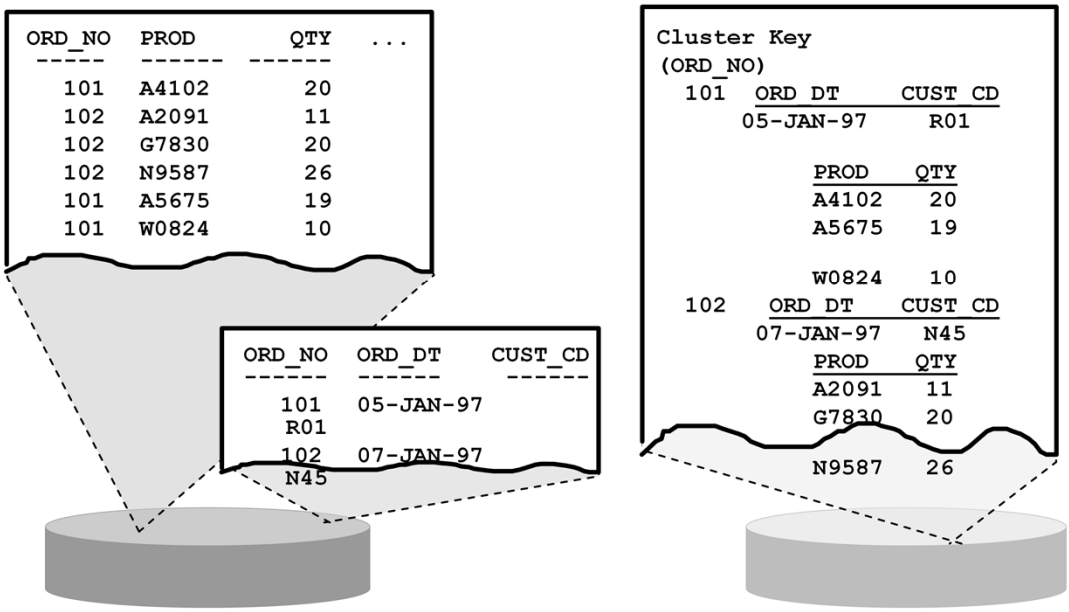
Индекс-таблицы отличаются от обычных таблиц только физической организацией. Логически операции с этими таблицами выполняются также, как и с обычными. индекс-таблица может указываться в командах `INSERT`, `SELECT`, `DELETE` и `UPDATE` также, как и обычная таблица.

Все опции команды `ALTER`, доступные при использовании обычных таблиц, могут быть применены и к индекс-таблицам. В их число входят: `ADD`, `MODIFY` и `DROP` для `COLUMNS`, а также `ADD` и `DROP` для `CONSTRAINTS`. Однако ограничение главного ключа для индекс-таблицы не может быть удалено, отложено (`deferred`) или

отключено (`disable`).

Кластеры

Типы таблиц
Секционирование
IOT
> Кластер
Задачи АБД



Некластеризованные таблицы `orders` и `order_item`

Кластеризованные таблицы `orders` и `order_item`

Copyright © 2006, Oracle. All rights reserved.

Определение кластеров

Кластер (*cluster*) – это группа из одной или нескольких таблиц, совместно использующих одни и те же блоки данных, поскольку эти таблицы содержат общие столбцы и часто используются в запросах с соединениями. Хранение таблиц в кластерах дает возможность АБД денормализовать данные, вид которых очевиден для конечного пользователя и программиста. Применение кластеров не требует изменения кода приложения, обращающегося к таблицам. Кластеры прозрачны для конечного пользователя и программиста.

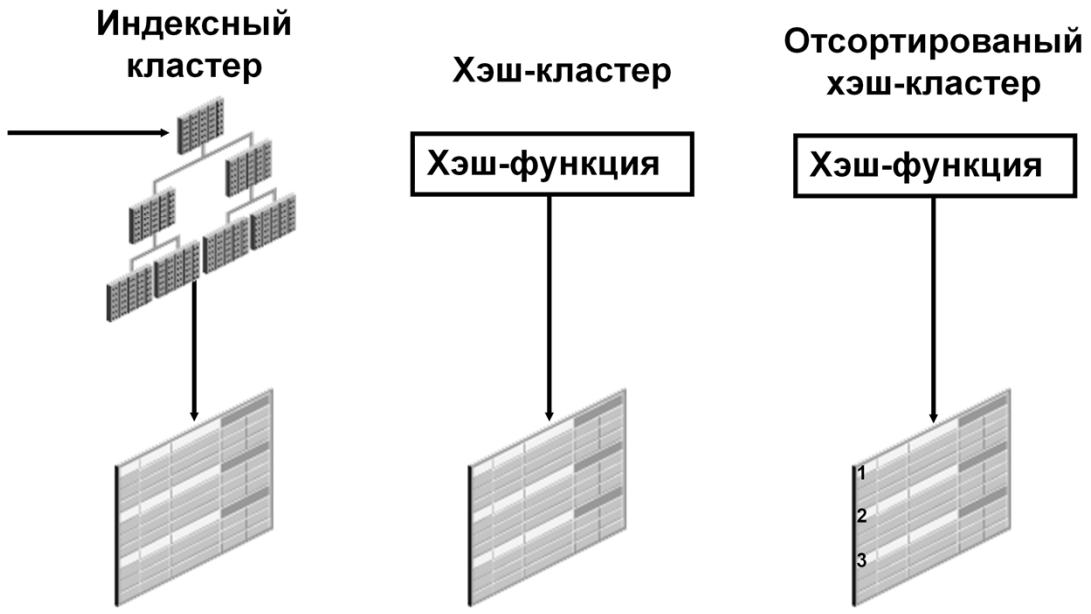
Преимущества кластеров

- Уменьшается дисковый ввод-вывод и время соединения кластеризованных таблиц.
- Значение каждого кластерного ключа хранится только один раз для всех строк с одинаковым значением ключа, поэтому для хранения используется меньше пространства.

Производительность

Полные просмотры для кластеризованных таблиц выполняются обычно медленнее, чем для некластеризованных.

Типы кластеров



ORACLE

10-14 Copyright © 2006, Oracle. All rights reserved.

Типы кластеров

Индексный кластер

Для сопровождения данных, содержащихся в *индексном кластере* (*index cluster*), используется *кластерный индекс (cluster index)*. Он должен быть доступен для хранения, извлечения и сопровождения данных в индексном кластере.

Кластерный индекс указывает на блок данных, содержащий строки с заданным значением ключа. Структура кластерного ключа похожа на структуру обычного индекса.

В отличие от обычного индекса кластерные индексы хранят неопределенные значения. При этом для каждого значения ключа кластерного индекса создается только один элемент индекса. Поэтому при одном и том же наборе ключевых значений кластерные индексы могут быть меньше, чем обычные.

Хэш-кластер

Для определения местоположения строки в *хэш-кластере* (*hash cluster*) при выборке и выполнении операций DML используется хэш-алгоритм (определяемый пользователем или сгенерированный системой).

Когда производится поиск с использованием кластерного ключа по условию равенства, хэш-кластер может обеспечить больший выигрыш производительности по сравнению с индексным кластером, т.к. просматривается только один сегмент (не нужен доступ к индексу).

Типы кластеров (продолжение)

Отсортированный хэш-кластер

Внутри *отсортированного хэш-кластера* (*sorted hash cluster*) строки находятся в списках отсортированных строк. Каждый список соответствует определенному хэш-значению ключевых столбцов, определяемому в соответствующем хэш-кластере. Внутри каждого списка строки хранятся упорядоченно на основе значений столбцов ключа сортировки, которые определяются в соответствующем отсортированном хэш-кластере. По умолчанию строки выбираются в таком же порядке, когда выполняется запрос из кластеризованной таблицы и в предикате используются столбцы хэш-ключа.

Преимущества отсортированных хэш-кластеров в том, что при их использовании не затрачивается время ЦП и приватная память для сортировки данных при выполнении запросов, в которых необходимо гарантировать определенный порядок возвращаемых строк.

Ситуации, в которых полезны кластеры

Критерий	Индекс	Хэш	Отсорт. хэш
Равномерное распределение значений ключа	X	X	X
Случайное распределение значений ключа		X	X
Редко обновляемые ключи	X	X	X
Часто соединяемые таблицы “общее-детали”	X		
Предсказуемое количество значений ключа		X	X
Запросы, использующие в предикате равенство по ключу		X	X
Данные выбираются в том же порядке, в котором они были вставлены			X

ORACLE

10-16 Copyright © 2006, Oracle. All rights reserved.

Ситуации, в которых полезны кластеры

Когда не рекомендуется использовать кластеры

- Запросы, в которых соединяются таблицы, в приложении выполняются редко, а изменения значений общих столбцов – часто. Изменение в строке значения кластерного ключа происходит дольше, чем в некластеризованной таблице, так как Oracle должен при этом перенести измененную строку в другой блок для сопровождении кластера.
- По одной из кластеризованных таблиц часто выполняется полный просмотр.

  Для хранения этой таблицы используется больше блоков по сравнению с отдельной таблицей, не входящей в кластер .
- Данные из всех таблиц с одинаковым значением кластерного ключа превышают один или два блока Oracle. Для доступа к одной строке в кластеризованной таблице сервер Oracle читает все блоки, содержащие строки с таким же значением.

Когда не рекомендуется использовать хэш-кластеры

- Если таблица постоянно растет или если приложение часто изменяет значения столбцов кластерного ключа.
- Если в приложении часто выполняются полные просмотры таблиц и нужно выделять достаточно большое пространство для хэш-кластера с учетом предполагаемого роста.

Использование хэш-кластеров и индексных кластеры необходимо предварительно тщательно спланировать. При их применении могут возникнуть дополнительные накладные расходы, отрицательно влияющие на производительность, когда выполняются такие крупные операции, как массовая (прямая) вставка и перестройка объектов.

Отсортированный хэш-кластер: обзор

- **Новая структура, используемая для хранения данных, отсортированных по столбцам, не входящим в первичный ключ:**
  - **Для кластерного ключа подсчитывается хэш-значение.**
  - **Строки, соответствующие конкретному значению кластерного ключа, сортируются на основе ключа сортировки.**
- **Используется для гарантированной выборки строк в определенном порядке без выполнения сортировки данных:**
  - **Строки возвращаются в возрастающем или убывающем порядке для конкретного значения кластерного ключа.**
  - **Необязательно указывать предложение `ORDER BY` для выборки строк в возрастающем порядке.**

ORACLE

10-17 Copyright © 2006, Oracle. All rights reserved.

Отсортированный хэш-кластер: обзор

При хранении в БД Oracle данных обычной неупорядоченной таблицы строки не сортируются в определенном порядке, контролируемом пользователем. Более того, решение, где разместить строку, зависит от эвристических правил хранения. База данных Oracle не гарантирует возврат строк в определенном порядке, если только запрос не включает предложение `ORDER BY`.

Внутри отсортированного хэш-кластера создаются списки отсортированных строк. Каждый список соответствует конкретному хэш-значению столбцов ключа. Вместе с таким ключом определяется ключ сортировки хэш-кластера. Список задает порядок возврата строк по запросу к таблице, в котором в предикате используются столбцы хэш-ключа.

Отсортированные хэш-кластеры уменьшают использование времени ЦП и понижают потребность в частной памяти, возникающей при сортировке данных во время выполнения запросов, требующих гарантированного порядка возврата строк в различных командах SQL.

При запросе данных из хэш-кластеризованной таблицы по ключу кластеризации с предложением `ORDER BY`, содержащем только столбцы ключа сортировки или префиксные столбцы этого ключа, оптимизатор исключает накладные расходы сортировки, так как строки возвращаются отсортированными в соответствие с столбцами ключа. Однако дополнительная сортировка требуется для запросов, в которых в предложении `ORDER BY` указываются столбцы суффикса ключа сортировки или столбцы, не входящие в ключ сортировки. Такая дополнительная сортировка необходима, если отсутствуют подходящие индексы таблицы. По этой причине, создавая отсортированный хэш-кластер, тщательно

подбирайте столбцы ключа, определяющего порядок хранения и выборки строк.

Отсортированный хэш кластер: пример

```
CREATE CLUSTER calls_cluster
( origin_number  NUMBER          ← Ключ кластера
, call_timestamp NUMBER SORT     } ← Ключ
, call_duration  NUMBER SORT)    }   сортировки
HASHKEYS 10000
SINGLE TABLE HASH IS origin_number
SIZE 50;
```

```
CREATE TABLE calls
( origin_number  NUMBER
, call_timestamp NUMBER
, call_duration  NUMBER
, other_info     VARCHAR2(30))
CLUSTER calls_cluster(
origin_number,call_timestamp,call_duration
);
```

ORACLE

Copyright © 2006, Oracle. All rights reserved.

Отсортированный хэш кластер: пример

Метод определения *отсортированного хэш-кластера* (*sorted hash cluster*) очень похож на метод создания *хэш-кластера* (*hash cluster*).

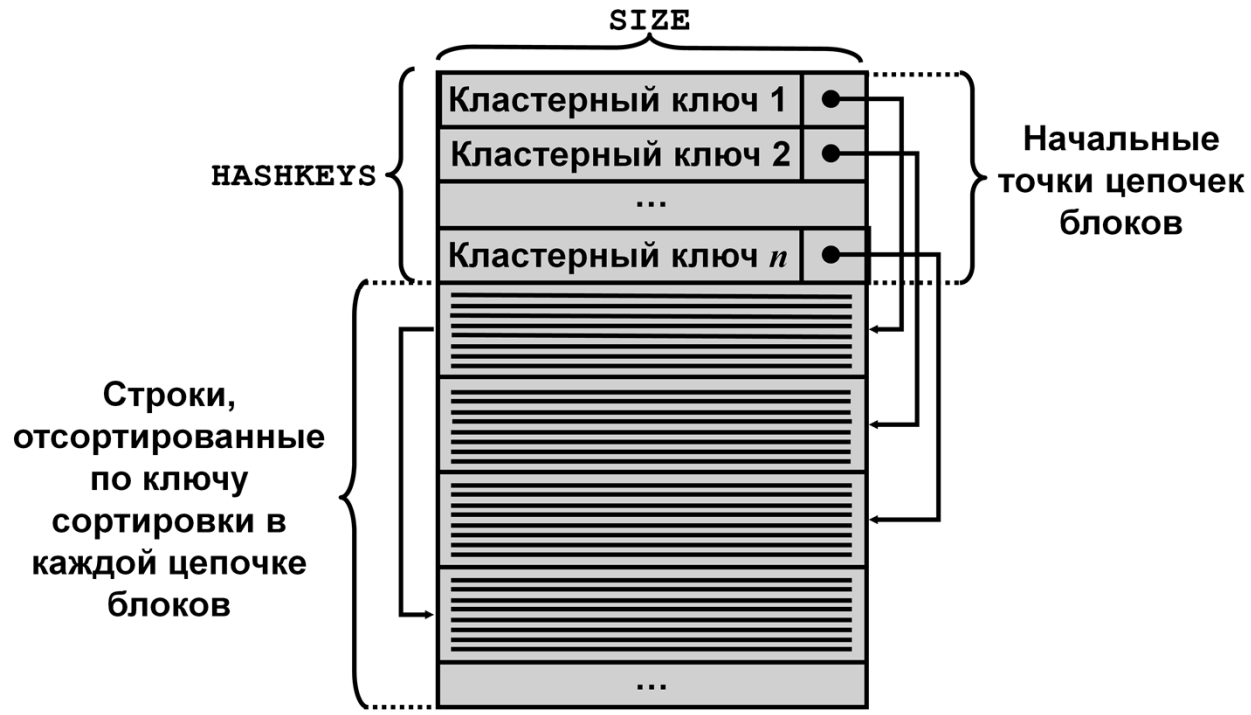
Сначала создается отсортированный хэш-кластер. Основное отличие от традиционных хэш-кластеров состоит в том, что дополнительно к столбцам ключа кластера определяются столбцы *ключа сортировки* (*sort key*). В приведенном на слайде примере `ORIGIN_NUMBER` – ключ кластера, а `CALL_TIMESTAMP` и `CALL_DURATION` –. ключи сортировки. Другое отличие в том, что в отсортированном хэш-кластере параметр `SIZE` определяет, сколько записей с метаданными хранится для конкретного значения хэш-ключа. Размер *записи с метаданными* (*metadata entry*) зависит в основном от размера столбцов кластерного ключа.

На втором шаге создается сама таблица, связываемая с отсортированным хэш-кластером с помощью фразы `CLUSTER`. Столбцы ключа кластера должны быть указаны в правильном порядке, за которыми должны следовать столбцы ключа сортировки.

В примере на слайде отражается следующий сценарий. Телекоммуникационной компании необходимо хранить записи исходящих звонков, соответствующие фиксированному набору телефонных номеров, проходящих через определенный коммутатор. С каждого телефонного номера может быть произведено неограниченное количество исходящих телефонных звонков. Данные о звонках хранятся в том порядке, как они были сделаны. Позже они обрабатываются в порядке “первым зафиксирован, первым выбран” для подготовки счета по каждому телефонному номеру. Каждый звонок определяется числовой временной меткой.

Примечание: в примере используется одна таблица, однако в кластере можно хранить и несколько таблиц.

Отсортированный хэш кластер: общая структура



Copyright © 2006, Oracle. All rights reserved.

Отсортированный хэш кластер: общая структура

В примере на предыдущем слайде в отсортированном хэш-кластере хранится только одна таблица. Хэш-функция очень простая и никаких противоречий не предвидится, так как сама функция определена на основе ключа кластеризации, а каждое значения кластерного ключа уникально. Параметр `HASHKEYS` фактически отражает количество различных исходящих телефонных номеров, а параметр `SIZE` – количество байтов, используемых для хранения метаданных каждого кластерного ключа.

Как вы можете видеть, первая часть сегмента отсортированного хэш-кластера зарезервирована для хранения *точек входа с метаданными* (*metadata entry*). Каждая такая точка входа содержит только ссылку на список соответствующих строк. Каждый список строится из ряда связанных друг с другом блоков Oracle. Сортировка каждого списка выполняется в соответствие с столбцами ключа сортировки.

Всякий раз, когда вы хотите выбрать строки для соответствующего значения кластерного ключа, находится хэш-значение, указывающее на расположение точки входа в метаданные, которые предоставляют доступ к отсортированному списку выбираемых строк.

Задачи сопровождения схем

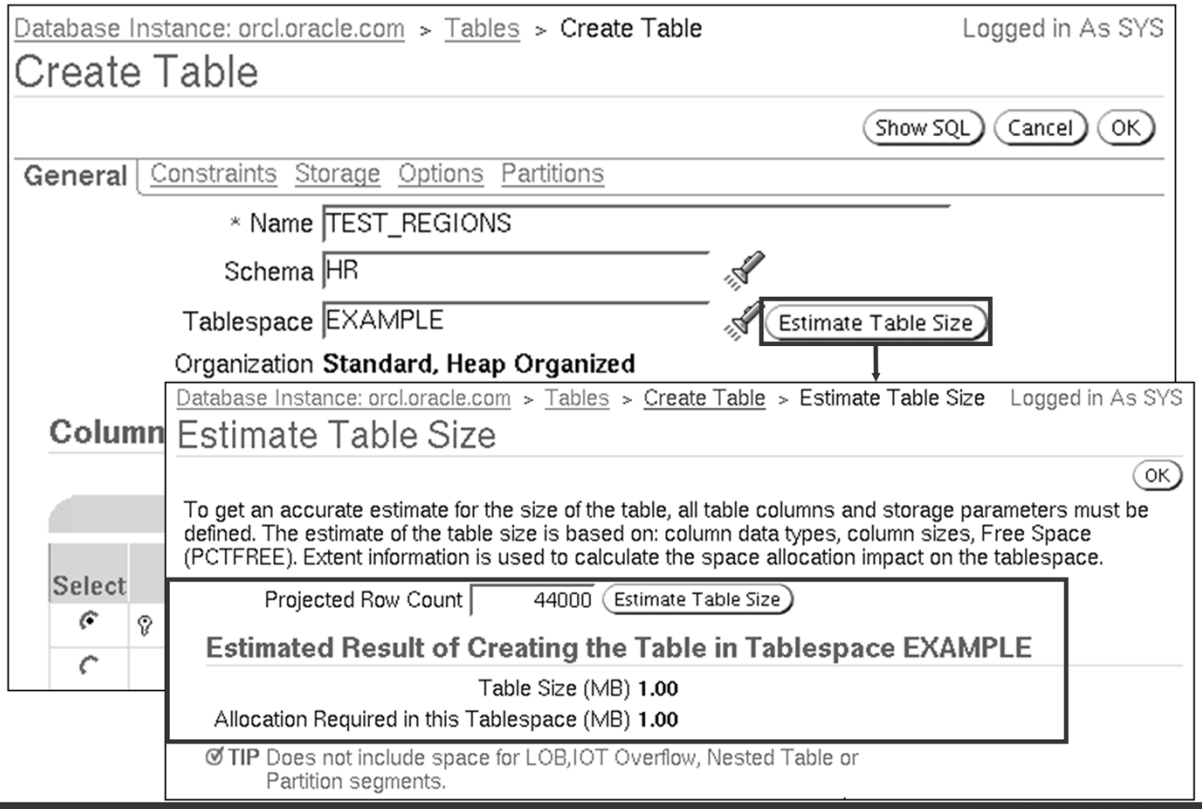
В круг задач АБД входят:

- **Оценка размера новых таблиц**
- **Анализ тенденций роста**
- **Сопровождение статистик оптимизатора**
- **Оперативная реорганизация объектов схем**

ORACLE

Copyright © 2006, Oracle. All rights reserved.

Оценка ресурсов, необходимых сегменту



10-21 Copyright © 2006, Oracle. All rights reserved.

Оценка ресурсов, необходимых сегменту

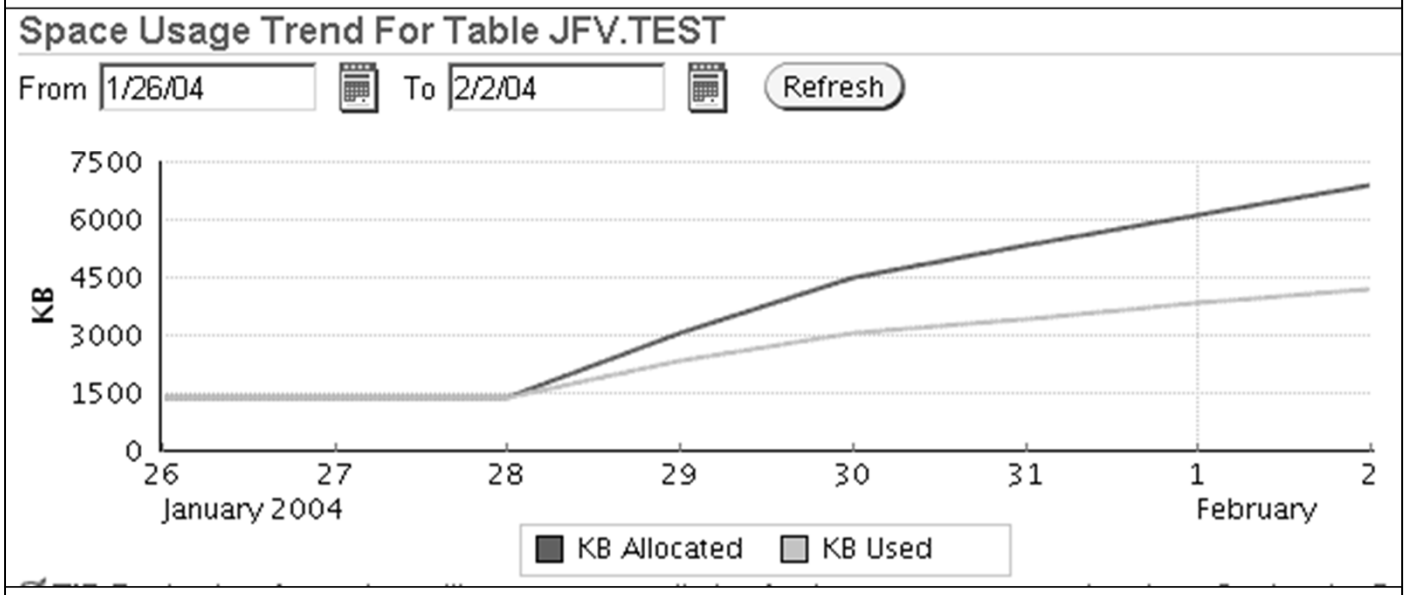
Возможность оценки позволяет приблизительно подсчитать объем ресурсов, требуемых при создании нового сегмента. Исходя из структуры таблицы или индекса и оценки количества строк в таблице, Oracle приблизительно подсчитывает размер дискового пространства, который вероятно потребуется объекту.

Например, оценка размера таблицы основывается на следующей информации: типы данных столбцов, размер столбцов и значение параметра `PCTFREE`. При подсчете требуемой памяти в выбранном табличном пространстве также учитывается информация об экстентах .

Чтобы воспользоваться этой возможностью, выберите на странице Administration ссылку Tables в секции Storage, а затем щелкните на кнопке Create для создания новой таблицы. Создать новую таблицу можно также, используя операцию “`Create like`”. На странице Create Table в секции Columns задайте типы данных столбцов. Также укажите, в каком табличном пространстве создается новая таблица, а затем щелкните на кнопке Estimate Table Size. Откроется страница Estimate Table Size, на которой можно задать предполагаемое число строк (Projected Row Count). Затем щелкните на кнопке Estimate Table Size, чтобы получить оценку размера таблицы и соответствующего места, выделяемого в табличном пространстве.

Анализ тенденций роста

- **Используется сегментным консультантом**
- **Собираемые статистики использования пространства хранятся в AWR**



ORACLE

Copyright © 2006, Oracle. All rights reserved.

Анализ тенденций роста

Информация о росте сегментов содержит важные сведения для консультантов и инструментальных средств подготовки отчетов. Например, эти данные используются сегментным консультантом, а также при подготовке графика тенденций роста сегмента.

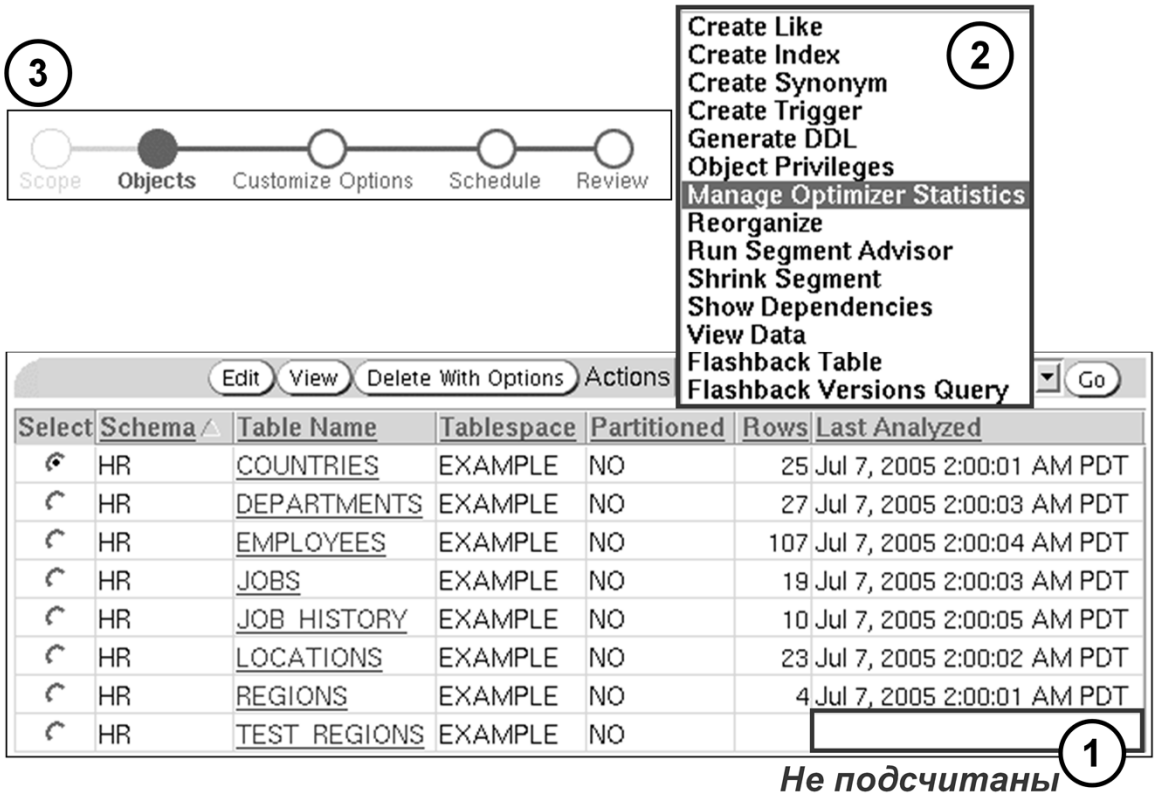
Некоторые характеристики возможности получения отчетов о тенденциях роста:

- Анализ данных репозитария рабочей загрузки. Устойчивые статистики использования пространства собираются в репозитарии рабочей загрузки. Они сохраняются в определяемой системой схеме. Эти статистики получаются во время создания моментальных снимков, а также когда проверяется состояние сигналов.
- Отражение тенденций роста в прошлом и моделирование их роста в будущем.
- Поддерживаются только табличные пространства с локальным управлением.

Отчет о тенденциях роста отражает динамику изменения размера сегмента в прошлом и предсказание его роста в будущем. Графически показываются прошлые данные об использовании пространства и линейный прогноз их роста.

В Database Control для получения отчета о тенденциях роста щелкните на ссылке Tables в секции Schema, расположенной на странице Administration. Затем выберите таблицу и щелкните на кнопке Edit. На странице редактирования таблицы перейдите на закладку Segments. В результате будет выведен отчет о тенденциях роста, пример которого показан на слайде. Пользователь может задать анализируемый период и получить для него график. В приводимом примере видна рассчитанная точка во времени, когда сегменту возможно потребуется больше пространства.

Сопровождение статистик оптимизатора



Copyright © 2006, Oracle. All rights reserved.

Сопровождение статистик оптимизатора

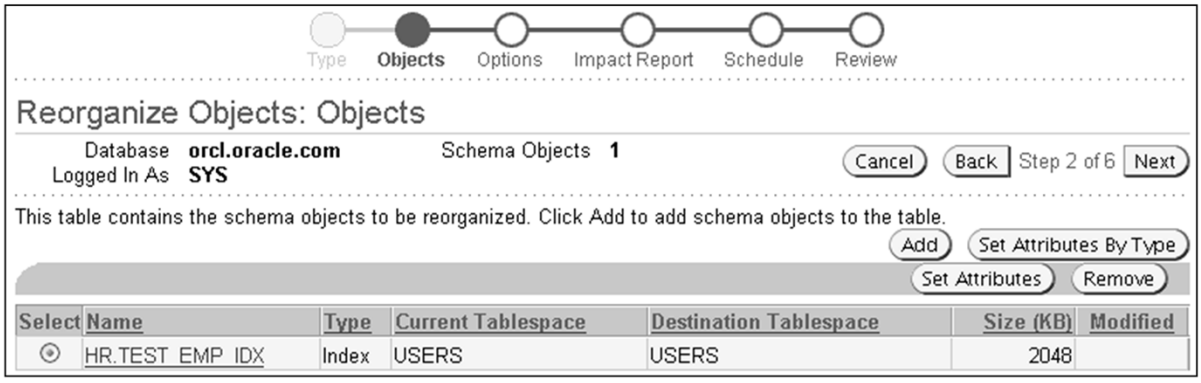
Настоятельно рекомендуется собирать статистики после больших операций DML, например, после вставки 9000 строк.

Пример, приведенный на слайде, показывает (1), что для таблицы `TEST_REGIONS` еще не были проанализированы ее статистики.

В Enterprise Manager перейдите следующим образом: Administration > Tables. Выберите “Manage Optimizer Statistics” в выпадающем списке поля Actions (2), а затем с помощью мастера создайте задание сбора статистик оптимизатора.

Оперативная реорганизация объектов схем

- **Модификация логической и физической структур объекта схемы, например, таблицы или индекса**
- **Прозрачна для пользователей**
- **Требуется пространство**



Copyright © 2006, Oracle. All rights reserved.

Оперативная реорганизация объектов схем

Иногда может возникнуть необходимость изменения логической или физической структуры таблицы, вызванная проблемами управления пространства, необходимостью повышения производительности или учета требований новых приложений. Для решения этой задачи при условии, что таблица остается доступной как для запросов, так и команд DML, используется мастер Reorganize Objects, предоставляемый Enterprise Manager, или пакет `DBMS_REDEFINITION`.

Реорганизация таблиц обеспечивает существенное увеличение доступности по сравнению с традиционными методами переопределения таблиц. Таблица блокируется в монопольном режиме только в течение очень небольшого временного окна, независимого от размера таблицы и сложности процесса переопределения. Вся операция полностью прозрачна для пользователей.

Процессу реорганизации необходимо пространство, приблизительно равное размеру занимаемого пространства переопределяемой таблицы. Когда добавляются новые столбцы, может потребоваться дополнительное пространство.

Оперативная реорганизация может использоваться для:

- перестроения фрагментированных индексов;
- перестроения фрагментированных таблиц;
- перемещения объектов в другое табличное пространство;
- пересоздания объектов с лучшими атрибутами хранения.

Оперативная реорганизация объектов схем (продолжение)

Опции операции реорганизации:

Type Objects **Options** Impact Report Schedule More

Reorganize Objects: Options

Database **orcl.oracle.com** Schema Objects **1**

Logged In As **SYS**

Cancel Back Step 3 of 6 Next

Method

Some object types can be reorganized online. With an online reorganization the objects have higher availability but the reorganization is slower. Do you want the reorganization to favor speed or availability?

- (•) Speed (offline) - object availability is not a concern
- ( ) Availability (online) - object availability is important
  - [ ] Use ROWID method - adds a hidden column to tables

Scratch Tablespace

Reorganizations are performed inside the database and require sufficient free space. The scratch tablespace is used for intermediate storage of objects during reorganization.

- (•) Use current tablespace
- ( ) Use scratch tablespace

Tablespace [ ]

Hide Advanced Options

Object Parameters

- [ ] Use parallel execution when possible
  - Parallel Degree (•) Default ( ) Value [ ]
- [ ] Rebuild indexes without logging for faster reorganization
- [x] Update any existing cost-based optimizer statistics
  - (•) Compute statistics based on all the rows of the selected objects
  - ( ) Estimate statistics based on some of the rows of the selected objects. This method is faster but the statistic is less accurate.
    - Estimate Percentage (•) Default - Oracle determines the best sample size for good statistics
    - ( ) Percentage [ ]
- [ ] Specify number of rows inserted between commits when reorganizing tables with LONG columns
  - Number of Rows 100

Session Parameters

Sort Area Size (KB) 64

(Sort area size has a significant impact on the performance of index creation.)

Реорганизация объектов: отчет о воздействии этой операции

Type Objects Options **Impact Report** Schedule More

Reorganize Objects: Impact Report

Database **orcl.oracle.com** Schema Objects **1**
Logged In As **SYS**

Cancel Back Step 4 of 6 Next

Script Generation Summary

Most Serious Message Severity **INFORMATION**
Generation Started **Aug 21, 2005 5:48:58 PM**
Generation Completed **Aug 21, 2005 5:49:01 PM**

Script Generation Information

The following table provides information about the objects and resources examined during script generation and lists details of any warnings or errors detected.

Object Name	Object Type	Message Severity	Message Type	Message
USERS	TABLESPACE	INFORMATION	Plan	Sufficient free space in Tablespace USERS. Starting Freespace with automatic extension: 33544176KB. Ending Freespace: 33546160KB. Lowest Freespace: 33544112KB.
HR	USER	INFORMATION	Plan	Sufficient tablespace quota for User HR.

Printable Page

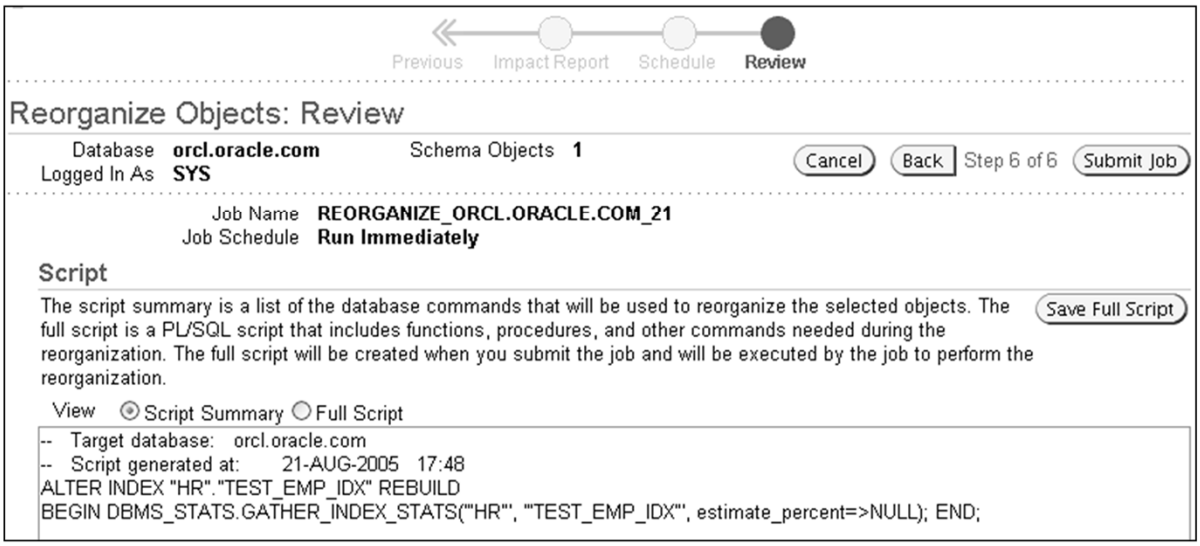
ORACLE

10-26 Copyright © 2006, Oracle. All rights reserved.

Реорганизация объектов: отчет о воздействии этой операции

Перед выполнением каких-либо команд предоставляется отчет, содержащий ошибки и предупреждения, а также обзор запланированных действий, например, действия по перестройке индекса.

Реорганизация объектов: обзор



Copyright © 2006, Oracle. All rights reserved.

Реорганизация объектов: обзор

Для получения полных сведений о командах, которые будут выполнены, выберите Full Script, а затем просмотрите команды запланированной операции. Чтобы получить только обзор операции, выберите Script Summary.

Основные шаги оперативной реорганизации, выполняемой вручную

1. Убедитесь в том, что таблица является подходящим кандидатом для оперативной реорганизации.
2. Создайте промежуточную таблицу.
3. Запустите процесс переопределения.
4. Скопируйте зависимые объекты (в результате для промежуточной таблицы автоматически создаются триггеры, индексы, права доступа и ограничения).
5. Выполните запрос к `DBA_REDEFINITION_ERRORS` для проверки наличия ошибок.
6. Дополнительно выполните синхронизацию для промежуточной таблицы.
7. Завершите переопределение.
8. Удалите промежуточную таблицу.

ORACLE

10-28 Copyright © 2006, Oracle. All rights reserved.

Основные шаги оперативной реорганизации, выполняемой вручную

Используемые команды и процедуры:

1. `DBMS_REDEFINITION.CAN_REDEF_TABLE`
2. `CREATE TABLE ...`
3. `DBMS_REDEFINITION.START_REDEF_TABLE`
4. `DBMS_REDEFINITION.COPY_TABLE_DEPENDENTS` и `DBMS_REDEFINITION.CONS_ORIG_PARAMS`
5. `SELECT object_name, base_table_name, ddl_txt FROM DBA_REDEFINITION_ERRORS;`
6. `DBMS_REDEFINITION.SYNC_INTERIM_TABLE`
7. `DBMS_REDEFINITION.FINISH_REDEF_TABLE`
8. `DROP TABLE ... PURGE`

Итоги

- Использование таблиц определенных типов в соответствие с вашими требованиями: обычных, секционированных, IOT и кластеризованных.
- Выполнение задач АБД:
  - Оценка размера новых таблиц
  - Анализ тенденций роста
  - Сопровождение статистик оптимизатора
  - Оперативная реорганизация объектов схем

ORACLE

Copyright © 2006, Oracle. All rights reserved.

Обзор практического занятия: управление объектами схем

- Мониторинг использования пространства таблицами и индексами
- Сопровождение статистик оптимизатора
- Реорганизация таблицы и индекса

ORACLE

Copyright © 2006, Oracle. All rights reserved.